

Technical Spec: SaaS Multi-Tenant Teams Scheduler

Approach: Delegated Permissions (Universal Support)

1. Overview

Calendly-style platform: visitor requests meeting → admin checks staff availability → books Teams meeting on staff's behalf → reschedule/cancel/swap later — all via Microsoft Graph, delegated auth. Works for Enterprise, Small Business, and Personal (@outlook.com) accounts, no org-wide PowerShell policy.

2. Azure / Entra ID Setup

A. App Registration

1. Azure Portal → Entra ID → App Registrations → New registration.
2. **Supported account types: "Accounts in any organizational directory and personal Microsoft accounts"** (multi-tenant + personal).
3. Authentication → Add platform (Web) → set redirect URI.
4. Generate Client Secret (or cert). Store `client_id` / `client_secret` in a secrets manager, not code/env.
5. API Permissions → Microsoft Graph → **Delegated**: `offline_access`, `Calendars.ReadWrite`, `OnlineMeetings.ReadWrite`, `User.Read`.

B. Why delegated, not app-only

App-only (client credentials) needs a Teams PowerShell **Application Access Policy** per staff/group before `isOnlineMeeting: true` works for anyone but the app — and it can't work for personal accounts at all (no tenant to grant a policy in). Delegated skips this: token is the staff member, Graph treats calls as that user in Outlook/Teams directly. No admin consent step, no policy propagation delay. Trade-off: staff must consent once; you depend on their refresh token staying valid.

3. Authentication & Token Management

Each staff member connects once (OAuth consent). Backend stores encrypted `refresh_token` to act while offline.

A. Authorization Header (every Graph call)

```
Authorization: Bearer {ACCESS_TOKEN}
Content-Type: application/json
Prefer: outlook.timezone="UTC"
```

B. Token Refresh Flow (background logic)

- **Endpoint:** POST `https://login.microsoftonline.com/common/oauth2/v2.0/token`
- **Payload (form-encoded):**

```
client_id={ID}
client_secret={SECRET}
grant_type=refresh_token
refresh_token={STORED_REFRESH_TOKEN}
scope=offline_access Calendars.ReadWrite OnlineMeetings.ReadWrite User.Read
```

- **Response fields:** `access_token`, `refresh_token` (rotates — update DB immediately), `expires_in` (~3600s).
- Cache `access_token`; refresh proactively before expiry, not per-call.
- `invalid_grant` = revoked. Mark staff `disconnected`, prompt re-consent — don't retry silently.

4. Core Workflow

All calls use the **target staff member's own token** (acts as `/me`).

Step 1: Check Availability

- **Endpoint:** POST `https://graph.microsoft.com/v1.0/me/calendar/getSchedule`
- **Headers:** `Authorization: Bearer {token}`, `Content-Type: application/json`, `Prefer: outlook.timezone="UTC"`
- **Request Payload:**

```
{
  "schedules": ["staff_email@domain.com"],
  "startTime": { "dateTime": "2026-09-05T09:00:00", "timeZone": "UTC" },
  "endTime": { "dateTime": "2026-09-05T18:00:00", "timeZone": "UTC" },
  "availabilityViewInterval": 30
}
```

Response fields:

```
{
  "value": [{
    "scheduleId": "staff_email@domain.com",
    "availabilityView": "002200110",
    "scheduleItems": [ { "start": {}, "end": {}, "status": "busy" } ]
  }]
}
```

`availabilityView` digits: 0 =free, 1 =tentative, 2 =busy, 3 =OOF, 4 =working elsewhere. `scheduleItems` gives busy-block detail.

Step 2: Create Teams Meeting

- **Endpoint:** POST `https://graph.microsoft.com/v1.0/me/events`
- **Headers:** Authorization: Bearer {token}, Content-Type: application/json, Prefer: outlook.timezone="UTC"
- **Request Payload:**

```
{
  "subject": "SaaS Intro Meeting",
  "body": { "contentType": "HTML", "content": "Discussion about organization inquiry." },
  "start": { "dateTime": "2026-09-05T14:00:00", "timeZone": "UTC" },
  "end": { "dateTime": "2026-09-05T14:30:00", "timeZone": "UTC" },
  "isOnlineMeeting": true,
  "onlineMeetingProvider": "teamsForBusiness",
  "attendees": [
    { "emailAddress": { "address": "visitor@client.com", "name": "Visitor" }, "type": "required" }
  ]
}
```

Response fields (persist `id` and `onlineMeeting.joinUrl`):

```
{
  "id": "AAMkAGI2...",
  "subject": "SaaS Intro Meeting",
  "start": { "dateTime": "2026-09-05T14:00:00.0000000", "timeZone": "UTC" },
  "end": { "dateTime": "2026-09-05T14:30:00.0000000", "timeZone": "UTC" },
  "organizer": { "emailAddress": { "address": "staff_email@domain.com" } },
  "attendees": [ { "emailAddress": { "address": "visitor@client.com" }, "type": "required" } ],
  "isOnlineMeeting": true,
  "onlineMeeting": { "joinUrl": "https://teams.microsoft.com/l/meetup-join/..." },
  "webLink": "https://outlook.office365.com/owa/?itemid=...&path=/calendar/item"
}
```

Step 3: Reschedule

- **Endpoint:** PATCH `https://graph.microsoft.com/v1.0/me/events/{event_id}`
- **Headers:** Authorization: Bearer {token}, Content-Type: application/json, Prefer: outlook.timezone="UTC"
- **Request Payload (only changed fields):**

```
{
  "start": { "dateTime": "2026-09-06T10:00:00", "timeZone": "UTC" },
  "end": { "dateTime": "2026-09-06T10:30:00", "timeZone": "UTC" }
}
```

Response: 200 OK, full updated event (same shape as Step 2) — `id` and `onlineMeeting.joinUrl` unchanged, `start` / `end` reflect new time. Attendees auto-notified by Graph.

DB: update `requested_start` / `requested_end`, `status` = Rescheduled.

Step 4: Cancel / Delete

Cancel (preferred, notifies visitor):

- **Endpoint:** `POST /me/events/{event_id}/cancel`
- **Headers:** `Authorization: Bearer {token}`, `Content-Type: application/json`
- **Request Payload:** `{ "comment": "Apologies, this meeting has been cancelled." }`
- **Response:** `202 Accepted`, empty body.

Delete (no notification):

- **Endpoint:** `DELETE /me/events/{event_id}`
- **Headers:** `Authorization: Bearer {token}`
- **Response:** `204 No Content`, empty body. Use only if you send your own cancellation notice.

DB: `status = Cancelled`, keep row for audit, record `cancelled_at` / `cancelled_by`.

Step 5: Swap Host (Person A → Person B)

No shared service-account calendar in this model (personal accounts have none) — each staff member is organizer of their own event, and an event's organizer can't be changed. Cancel-and-rebook required:

1. **Cancel:** `POST /me/events/{event_id}/cancel` using **Person A's** token (payload/response as Step 4).
2. **Re-book:** `POST /me/events` using **Person B's** token (payload/response as Step 2) → new `event_id`, new `joinUrl`.
3. **Notify:** system sends visitor the new `joinUrl` directly — Graph's own notice only covers the cancellation, no shared event to auto-carry the new link.

DB: old row → `status = Cancelled`; new row → `status = Swapped`, new `ms_event_id` / `join_url`, `staff_email` = Person B.

5. Admin Experience

1. **Staff connection:** staff log in via Microsoft OAuth once; backend captures `refresh_token`.
2. **Admin dashboard:** lists "Connected" staff and disconnected ones needing re-auth.
3. **Cross-account action:** Admin clicks "Schedule for Staff A" → backend fetches Staff A's `refresh_token` → refreshes (§3B) → calls Graph `/me/...` as Staff A.
4. **Universal compatibility:** `@bank.com` (enterprise), `@agency.onmicrosoft.com` (small biz), `@outlook.com` (freelancer) — same code path, no per-tenant policy step.

6. Summary of Parameters & Responses

Parameter	Type	Importance
event_id	String	Primary Key. Store to edit/cancel/delete the meeting.
joinUrl	URL	Teams link for visitor and staff.
refresh_token	Token	Long-term key; encrypted in DB, lets admin book while staff offline.
isOnlineMeeting	Boolean	Must be <code>true</code> to generate the Teams link.

Action	Method	Endpoint	Notifies attendees?
Reschedule	PATCH	<code>/me/events/{id}</code>	Yes (automatic)
Cancel	POST	<code>/me/events/{id}/cancel</code>	Yes (with comment)
Delete	DELETE	<code>/me/events/{id}</code>	No
Swap host	cancel + POST	<code>/me/events/{id}/cancel</code> then <code>/me/events</code> (new token)	Cancel: yes. New booking: yes (new invite)

7. Error Handling & Reliability

- **429 Too Many Requests:** respect `Retry-After` , back off/retry.
- **401 / `invalid_grant` on refresh:** token revoked — mark staff disconnected, prompt re-consent, don't retry silently.
- **404 on event:** already deleted/moved by staff directly in Outlook — handle gracefully in reschedule/cancel/swap.
- Consider Graph change notifications (webhooks) on calendar resources so direct Outlook/Teams edits don't leave DB stale — fast-follow, not v1-blocking.

8. Database Schema

Field	Notes
visitor_email / visitor_name	for notifications
requested_start / requested_end	from intake, before host assigned
status	Requested → Scheduled → Rescheduled → Swapped → Cancelled (any → Cancelled)
cancelled_at / cancelled_by	audit trail; keep row, don't hard-delete
staff_email	current host
ms_event_id	Graph event ID, needed for all PATCH/DELETE/cancel calls
join_url	Teams link shown to visitor
refresh_token	encrypted, per-staff, long-term key
access_token / token_expires_at	cached, refreshed proactively
account_type	enterprise / small_business / personal — informational only, same code path

9. Security Requirements

- refresh_token encrypted at rest (Key Vault/KMS-backed), never logged.
- Scopes: offline_access , Calendars.ReadWrite , OnlineMeetings.ReadWrite , User.Read — no .All /app-only grants.
- App Registration: "Accounts in any organizational directory and personal Microsoft accounts."
- Always store/send times in UTC; convert for display only.

10. Open Decision

Swap-host loses "same joinUrl" continuity (no service-account model) since each staff member organizes their own event. If org-tier customers need same-link swaps, that needs a shared service-account calendar for those tenants specifically — breaks personal-account universality. Flag as per-tier product decision, not blocking v1.